Sistemas operativos

Práctica 1: Introducción a la Shell, Procesos e Hilos

Memoria

Autores: Claudia Saiz Escribano y Duna Puente Romera

Grupo de laboratorio: 2201

Fecha: 6 de marzo de 2026

Tabla de contenidos

1. Objetivo 3
2. Requisitos 3
3. Pruebas realizadas 4
4. Cuestiones del enunciado 7

1. Objetivo

La primera práctica de sistemas operativos ha consistido en una breve introducción a la gestión y manejo de las principales herramientas dentro de los procesos. Estas herramientas han consistido en el uso de la shell, la programación en C con POSIX, los procesos y ejecuciones de los programas, el empleo de los hilos y tuberías, y el tratamiento de ficheros.

Tras la realización de la práctica, podemos afirmar haber obtenido un mayor entendimiento de la aplicación de estos recursos, teniendo de antemano la base teórica proporcionada durante las clases.

2. Requisitos

Todos los requisitos que se incluyen a continuación han sido implementados con éxito. En primer lugar, el programa se asegura de recibir el número de parámetros solicitado y da un mensaje de error en caso contrario. También, se implementa el ejecutable con el nombre solicitado y el programa se divide en dos procesos fundamentales tras hacer el fork: el apartado del Minero (el proceso padre) y el apartado del Registrador (el proceso hijo). Por otro lado, se han establecido los mensajes de salida solicitados para cada proceso en función de su correcta o incorrecta ejecución.

A continuación, se ha desarrollado el proceso multihilo del Minero siguiendo las instrucciones del enunciado. Se divide el espacio de búsqueda en base a los hilos, asegurándonos de que se recorren todos los valores, se crean los hilos y se esperan a los mismos, y se ha implementado una verificación de manera que una vez se halle la solución, todos los hilos finalicen. Finalmente, se actualiza el nuevo objetivo para la siguiente ronda.

En el caso del Registrador, se han establecido las dos tuberías pertinentes para la comunicación con el Minero. Como se pide en las indicaciones, el Minero pasa al Registrador la solución y objetivo de cada ronda, éste lo refleja en un fichero y luego devuelve un mensaje de confirmación para continuar con la siguiente ronda. Para acabar, una vez lo dictamina el Minero, el Registrador finaliza su proceso e imprime su estatus de salida. Es necesario mencionar que, a la hora de imprimir la información por fichero, se emplea el formato requerido en esta práctica.

3. Pruebas realizadas

Se han realizado varias pruebas con diferentes números de rondas y de hilos, de forma que hemos podido comprobar el correcto funcionamiento del programa en todos los casos, además de analizar la eficiencia de la ejecución en función del número de hilos utilizados.

Primero, hemos empezado por ejecutar distintos casos de prueba para comprobar el correcto funcionamiento del programa. Hemos comprobado la salida de nuestro programa con los ejemplos de ejecución proporcionados, y hemos podido ver que concordaba el resultado. Además, no ocurren fugas de memoria como hemos podido ver con valgrind, y el documento con nombre del pid del proceso padre se ha generado correctamente.

```
Running miner
Solution accepted: 00000000 --> 08916551
Solution accepted: 08916551 --> 05970796
Solution accepted: 05970796 --> 00937352
Solution accepted: 00937352 --> 01835314
Solution accepted: 01835314 --> 09331340
Logger exited with status 0
Miner exited with status 0
```

```
==1717803== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyri
ght info
==1717803== Command: ./miner 0 5 3
==1717803==
Solution accepted: 00000000 --> 08916551
Solution accepted: 08916551 --> 05970796
Solution accepted: 05970796 --> 00937352
Solution accepted: 00937352 --> 01835314
Solution accepted: 01835314 --> 09331340
==1717804==
==1717804== HEAP SUMMARY:
==1717804==     in use at exit: 0 bytes in 0 blocks
==1717804==   total heap usage: 2 allocs, 2 frees, 4,568 bytes allocat
ed
==1717804==
==1717804== All heap blocks were freed -- no leaks are possible
==1717804==
==1717804== For lists of detected and suppressed errors, rerun with: -
s
==1717804== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 fro
m 0)
Logger exited with status 0
Miner exited with status 0
==1717803==
==1717803== HEAP SUMMARY:
==1717803==     in use at exit: 0 bytes in 0 blocks
==1717803==   total heap usage: 6 allocs, 6 frees, 1,936 bytes allocat
ed
==1717803==
==1717803== All heap blocks were freed -- no leaks are possible
==1717803==
==1717803== For lists of detected and suppressed errors, rerun with: -
s
==1717803== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 fro
m 0)
```

```
1717959.log
Id:       1
Winner:   1717959
Target:   00000000
Solution: 08916551 (validated)
Votes:    1/1
Wallets:  1717959:1

Id:       2
Winner:   1717959
Target:   08916551
Solution: 05970796 (validated)
Votes:    2/2
Wallets:  1717959:2

Id:       3
Winner:   1717959
Target:   05970796
Solution: 00937352 (validated)
Votes:    3/3
Wallets:  1717959:3

Id:       4
Winner:   1717959
Target:   00937352
Solution: 01835314 (validated)
Votes:    4/4
Wallets:  1717959:4

Id:       5
Winner:   1717959
Target:   01835314
Solution: 09331340 (validated)
Votes:    5/5
Wallets:  1717959:5
```

También ocurre todo correctamente si forzamos que una de las soluciones sea rechazada. En este caso implementamos simplemente que la salida 09331340 siempre sería incorrecta.

```
Running miner
Solution accepted: 00000000 --> 08916551
Solution accepted: 08916551 --> 05970796
Solution accepted: 05970796 --> 00937352
Solution accepted: 00937352 --> 01835314
Solution rejected: 01835314 --> 09331340
Logger exited with status 0
Miner exited with status 0
```

```
1718451.log
Id:       1
Winner:   1718451
Target:   00000000
Solution: 08916551 (validated)
Votes:    1/1
Wallets:  1718451:1

Id:       2
Winner:   1718451
Target:   08916551
Solution: 05970796 (validated)
Votes:    2/2
Wallets:  1718451:2

Id:       3
Winner:   1718451
Target:   05970796
Solution: 00937352 (validated)
Votes:    3/3
Wallets:  1718451:3

Id:       4
Winner:   1718451
Target:   00937352
Solution: 01835314 (validated)
Votes:    4/4
Wallets:  1718451:4

Id:       5
Winner:   1718451
Target:   01835314
Solution: 09331340 (rejected)
Votes:    5/5
Wallets:  1718451:5
```

El programa gestiona correctamente las situaciones de error, por ejemplo, cuando se manda un no permitido de rondas o hilos, o cuando no se envían los parámetros correctamente. También cuando por alguna razón uno de los procesos termina antes de tiempo.

```
Running miner
Error in the input parameters:

./miner < TARGET_INI > < ROUNDS > < N_THREADS >
make: *** [Makefile:27: run] Error 1
```

```
Running miner
N_THREADS must be greater than 0
make: *** [Makefile:27: run] Error 1
```

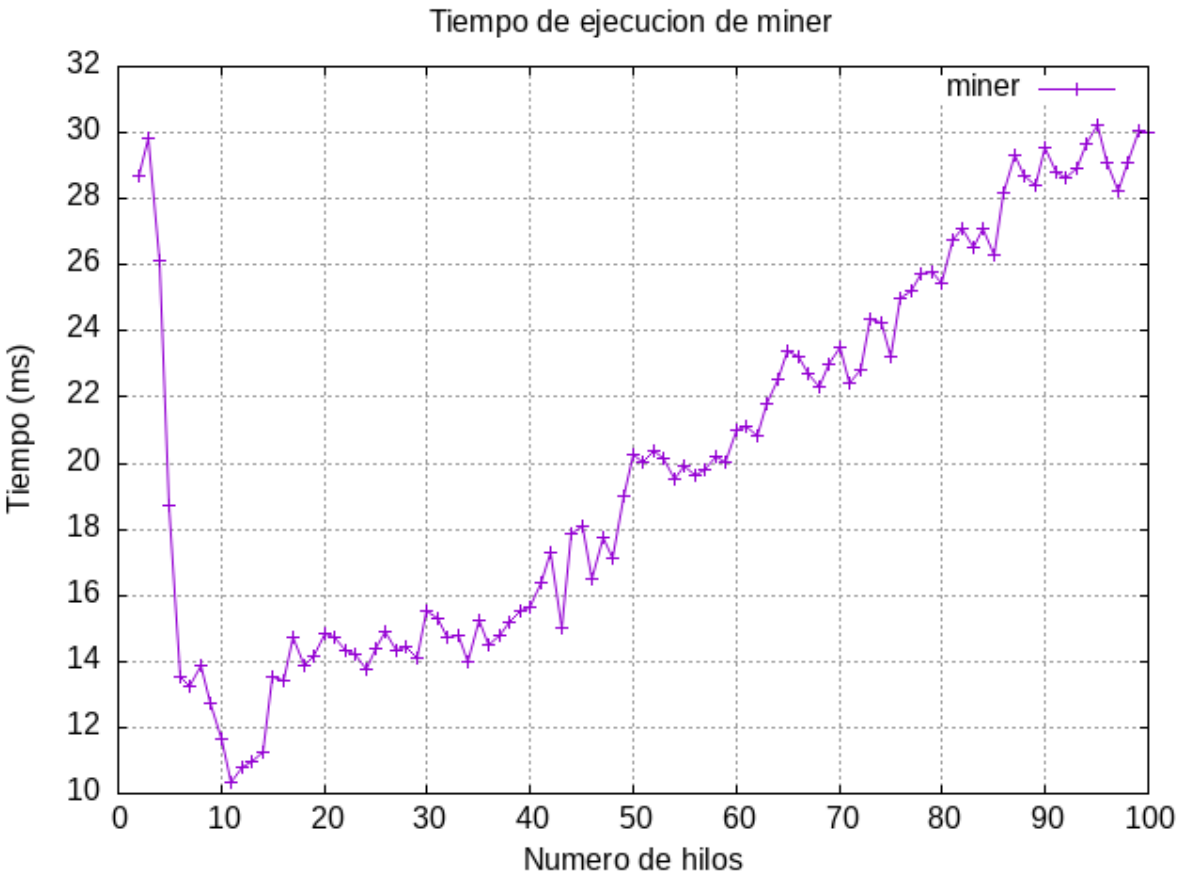
```
Running miner
ROUNDS must be greater than 0
make: *** [Makefile:27: run] Error 1
```

```
Running miner
TARGET must be positive integer
make: *** [Makefile:27: run] Error 1
```

4. Cuestiones del enunciado

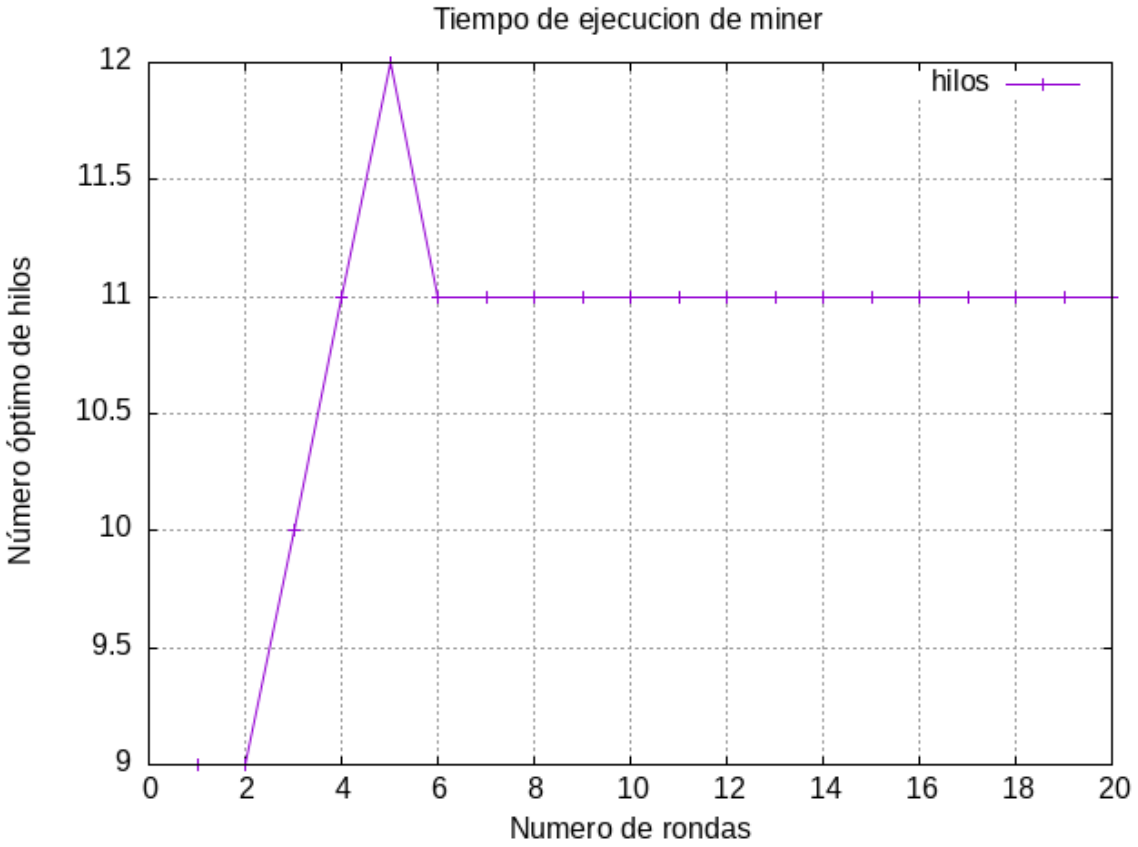
A continuación, queríamos analizar el rendimiento del programa, por lo que hemos observado primero como varían los tiempos de ejecución para un número de rondas fijo y variando el número de hilos. Después hemos analizado como cambian estos resultados para los diferentes números de rondas, siempre con el mismo target inicial (0).

Para ver como varían los tiempos de ejecución en función del número de hilos, creamos un script en bash, tiempos.sh, que mide los tiempos de ejecución para cada caso, y lo escribe en un fichero, a partir del cual se puede generar una gráfica en gnuplot.



Esta gráfica en concreto muestra el tiempo de ejecución del programa teniendo fijo target (0) y cinco rondas, pero con número de hilos aumentando desde 2 hasta 100. Podemos ver como el rendimiento mejora rápidamente al principio, hasta llegar a aproximadamente 12 hilos. A partir de ahí, empieza a empeorar, pero esta vez a una velocidad menor.

Esto se debe a que, al inicio, dividir la tarea entre varios hilos hace que se ejecute mucho más rápido. Sin embargo, llega un punto en el que el coste de crear hilos adicionales supera el tiempo ahorrado al dividir la tarea, lo que supone que según van aumentando el número de hilos, el tiempo de ejecución incremente.



Esta gráfica muestra el número óptimo de hilos para diferentes cantidades de rondas, desde 1 hasta 20. Se observa que, aunque en los primeros valores el número óptimo varía ligeramente, rápidamente se estabiliza en torno a 11 hilos. Además, a medida que aumentaba el número de rondas, la diferencia de tiempo entre el número óptimo y las demás se hacía cada vez mayor.

Podemos por lo tanto confirmar, que 11 hilos proporcionan el mejor rendimiento para este programa en este ordenador, sin embargo, este valor puede ser diferente en otros ordenadores, porque depende de la capacidad de procesamiento del ordenador con el que se esté ejecutando.